

Work Division Report

Parallel Distributed Computing Project Milestones 1, 2, and 3

Laraib Fatima Farooqui

l.masood.26521@khi.iba.edu.pk

I was responsible for implementing the sequential crawling algorithm in Milestone 1, which served as the single-process baseline that all parallel architectures were later compared against. I built the BFS crawler using `selectolax` for fast HTML parsing and made sure it correctly handled per-domain caps, URL normalisation, and concurrent batching.

For Milestone 2, I implemented the sequential PageRank baseline. This involved translating the core iterative formula into NumPy with double buffering to prevent partially-updated values from contaminating each sweep, and adding early stopping based on the L^∞ convergence criterion. This sequential implementation became the reference that all parallel speedups were measured against.

Beyond the code, I took the lead on report writing and results compilation across all three milestones. I collected benchmark numbers from the rest of the team, organised them into the tables and figures throughout this report, and wrote the majority of the prose covering experimental results and discussion.

Hamza Ahmed

h.ahmed.29265@khi.iba.edu.pk

I contributed across two milestones. In Milestone 1, I set up the `asyncio` infrastructure that underpins all three crawlers, including the event loop configuration, semaphore-based per-domain concurrency controls, and timeout handling, making sure these were consistent and correct across all architectures. I also implemented the Ray Peer-to-Peer crawler, building the `P2PWorker` actors with consistent-hash-based URL partitioning, the direct `receive()` routing between workers that bypasses any central queue, and the lightweight `P2PSignal` termination actor.

In Milestone 2, I implemented the Distributed Reduction PageRank strategy. Instead of collecting all chunk results at the driver, results are merged through a binary tree of Ray reduction tasks, which reduces per-iteration coordination cost. I also contributed to testing and analysis for Milestone 2, running the worker-scaling experiments.

Muhammad Hamza Arif

m.arif.29234@khi.iba.edu.pk

I designed and implemented the Ray Master-Worker crawler for Milestone 1. This involved building the centralised **MWMaster** Ray actor, which owns all shared crawl state including the BFS queue, the visited URL set, the graph, and per-domain counts. I also built the stateless **MWWorker** actors that fetch URLs asynchronously and submit discovered links back to the master. I tuned the worker concurrency settings and verified that centralising all state in the master correctly prevented any URL from being visited more than once.

I also led testing and analysis for Milestone 1, running the full hyperparameter sweep across all combinations of seed counts, crawl depths, and worker counts. I analysed the results, identified the performance crossover points, and wrote the final report sections covering the Master-Worker architecture design, the trade-off discussion, and the cross-architecture comparison.

Arbaaz Murtaza

a.murtaza.29052@khi.iba.edu.pk

For Milestone 2, I implemented the Centralised Aggregation parallel PageRank strategy. This involved partitioning the node space into equal-sized chunks, pushing the CSR graph arrays into the Ray object store via `ray.put()` for zero-copy reads, and collecting per-chunk rank results from all workers each iteration through `ray.get()` calls back to the driver. I benchmarked this strategy across all three graphs and recorded the performance results that appear in the results tables.

In Milestone 3, I implemented the sequential incremental update path and the micro-update algorithm. The micro-update propagates rank deltas outward through a small bounded frontier on a single CPU thread, requiring no Ray overhead and touching only a handful of nodes regardless of total graph size. I ran the accuracy validation experiments for Milestone 3, comparing incremental rank vectors against the full-recomputation baseline at each graph scale, and I wrote the analysis and testing sections for that milestone.